

SOFTWARE ARCHITECTURE – PRACTICAL TASK

Task 1. REAL-WORLD Examples of Architecture Patterns

1. Repository Architecture Pattern

- **Real-world Example:** IDEs (Integrated Development Environments) like Eclipse or VS Code, and Database-driven Web Applications (e.g., Banking Core Systems).
- **When is it most appropriate to use:**
 - When a system needs to process, store, and share large amounts of persistent data across multiple independent components.
 - When components need to interact or share state, direct component-to-component communication would create overly tight coupling.
- **Why is it most appropriate:**
 - **Decoupling:** Independent tools (like a code editor, syntax highlighter, debugger, and version control plugin) can all read from and modify the central project tree/ repository without needing to know about each other's inner workings.
 - **Data Integrity & Centralization:** It provides a single source of truth for complex data, making access control, backup, and transaction management much easier to maintain.

2. Client-Server Architecture Pattern

- **Real-world Example:** Web Browsers & Web Servers (E.g., accessing Google Search, Amazon, or the HyperionDev dashboard via Chrome).
- **When is it most appropriate to use:**
 - When resources, databases, or high-performance compute workloads need to be centralized on powerful server hardware while allowing many distributed users(clients) access over a network.
 - When client devices vary significantly in performance capability (e.g., mobile, phones, laptops, tablets).
- **Why is it most appropriate:**
 - **Resource Management:** It offloads heavy processing and storage requirements from client devices to dedicated server hardware.
 - **Centralized Control & Security:** Updates, security patches, and business logic can be updated on the server side instantly without requiring client application software.

3. Pipe and Filter Architecture Pattern

- **Real-world Example:** Unix/Linux Command- Line Pipelines (e.g., `cat access.log | grep "404" | wc -l`) or **ETL (Extract, Transform, Load)** Data pipelines (e.g., Apache Spark streaming telemetry data).
- **When is it most appropriate to use:**
 - When data needs to undergo a fixed sequence of independent, step-by-step transformations.

- When system tasks can easily be broken down into discrete processing stages (filters) connected by data streams (pipes).
- **Why is it most appropriate:**
 - **Reusability & Extensibility:** Individual filter components can be swapped, upgraded, or re-ordered easily without disrupting the rest of the processing pipeline.
 - **Parallel Processing:** Transformations can occur in parallel or in a continuous stream, making it highly efficient for data-driven applications handling raw data feeds.

Task 2. Examples of Combined Architecture Pattern

1. E-Commerce Platform (Client – Server + Repository Patterns)

- **Description:** A retail web application (like Amazon) uses a **Client-Server** model to handle incoming web traffic from millions of users across different devices. Behind the scenes, the backend service components (Inventory, Payment, Recommendations, Order Processing) interact through a centralized database via a Repository pattern.
- **Strengths:**
 - **Scalability for Clients:** The client-server layer easily scales to handle peak web traffic without altering underlying core database structures.
 - **Centralized State:** The central data repository ensures that stock numbers and order states are updated in real time across all front-end client sessions.
- **Limitations:**
 - **Single Point of Failure:** A failure or bottleneck at the central repository level impacts all connected services and client endpoints.
 - **Network Dependency:** Users require a stable network connection, and database latency directly impacts client responsiveness.

2. Video Streaming & Transcoding Service (Client-Server + Pipe and Filter Patterns)

- **Description:** Video Platforms (like YouTube or Netflix) use a client-server architecture to stream media to end users. When a video is uploaded, it passes through a Pipe and Filter pipeline (Raw File Upload → Audio/Video Extraction → Compression/Encoding → Multi-resolution Rendering → Storage).
- **Strengths:**
 - **High-Performance Data Processing:** The pipe and filter architecture processes video files in continuous parallel stages without overloading web server nodes.
 - **Clear Separation of Concerns:** Streaming logic (server-client handling) remains completely independent from background processing pipelines (encoding/ transcoding).
- **Limitations:**
 - **Increased Overhead:** Splitting streaming and processing into distinct patterns introduces systemic complexity, requiring robust inter-component communication channels.

- **Latency & Computational Cost:** Parsing and streaming data through multiple filters in sequence can increase processing time and consume substantial compute resources.